

NCERT Solutions For Class 12 Computer Science

Chapter 6 – Searching

1. Using linear search determine the position of 8, 1, 99 and 44 in the list:
[1, -2, 32, 8, 17, 19, 42, 13, 0, 44]

Draw a detailed table showing the values of the variables and the decisions taken in each pass of linear search.

Let $list1 = [1, -2, 32, 8, 17, 19, 42, 13, 0, 44]$, $n = 10$

(a) Searching for 8:

index	$index < n$	$list1[index] = key$	$index = index + 1$
0	$0 < 10?$ Yes	$1 = 8?$ No	1
1	$1 < 10?$ Yes	$-2 = 8?$ No	2
2	$2 < 10?$ Yes	$32 = 8?$ No	3
3	$3 < 10?$ Yes	$8 = 8?$ Yes	

Result: 8 is found at **position 4** (index 3).

(b) Searching for 1:

index	index < n	list1[index] = key	index = index + 1
0	0 < 10? Yes	1 = 1? Yes	

Result: 1 is found at **position 1** (index 0).

(c) Searching for 99:

index	index < n	list1[index] = key	index = index + 1
0	0 < 10? Yes	1 = 99? No	1
1	1 < 10? Yes	-2 = 99? No	2
2	2 < 10? Yes	32 = 99? No	3

index	index < n	list1[index] = key	index = index + 1
3	3 < 10? Yes	8 = 99? No	4
4	4 < 10? Yes	17 = 99? No	5
5	5 < 10? Yes	19 = 99? No	6
6	6 < 10? Yes	42 = 99? No	7
7	7 < 10? Yes	13 = 99? No	8
8	8 < 10? Yes	0 = 99? No	9

index	index < n	list1[index] = key	index = index + 1
9	9 < 10? Yes	44 = 99? No	10

Result: 99 is **not found** in the list.

(d) Searching for 44:

index	index < n	list1[index] = key	index = index + 1
0	0 < 10? Yes	1 = 44? No	1
1	1 < 10? Yes	-2 = 44? No	2
2	2 < 10? Yes	32 = 44? No	3
3	3 < 10? Yes	8 = 44? No	4
4	4 < 10? Yes	17 = 44? No	5

index	index < n	list[index] = key	index = index + 1
5	5 < 10? Yes	19 = 44? No	6
6	6 < 10? Yes	42 = 44? No	7
7	7 < 10? Yes	13 = 44? No	8
8	8 < 10? Yes	0 = 44? No	9
9	9 < 10? Yes	44 = 44? Yes	

Result: 44 is found at **position 10** (index 9).

2. Use the linear search program to search the key with value 8 in the list having duplicate values such as [42, -2, 32, 8, 17, 19, 42, 13, 8, 44]. What is the position returned? What does this mean?

The linear search will return **position 4 (index 3)** for the value 8, because it finds the first occurrence at that index. This means linear search returns the index of the first successful match, even if the value appears later again in the list.

3. Write a program that takes as input a list having a mix of 10 negative and positive numbers and a key value. Apply linear search to find whether the key is present in the list or not. If the key is present it should display the position of the key in the list otherwise it should print an appropriate message. Run the program for at least 3 different keys and note the result.

```
def linearSearch(list, key):  
    for index in range(0, len(list)):  
        if list[index] == key:  
            return index+1  
    return None  
  
list1 = [12, -3, 8, 0, -10, 25, -7, 19, 5, -2]  
keys_to_search = [8, -10, 21]  
  
for key in keys_to_search:  
    pos = linearSearch(list1, key)  
    if pos:  
        print(f"Number {key} is present at position  
{pos}")  
    else:  
        print(f"Number {key} is not present in the list")
```

Sample Output:

Number 8 is present at position 3
Number -10 is present at position 5
Number 21 is not present in the list

4. Write a program that takes as input a list of 10 integers and a key value and applies binary search to find whether the key is present in the list or not. If the key is present it should display the position of the key in the list otherwise it should print an appropriate message. Run the program for at least 3 different key values and note the results.

```
def binarySearch(arr, key):  
    first = 0  
    last = len(arr) - 1  
    while first <= last:  
        mid = (first + last) // 2  
        if arr[mid] == key:  
            return mid+1  
        elif key > arr[mid]:  
            first = mid + 1  
        else:  
            last = mid - 1  
    return None  
  
list1 = sorted([14, 5, 21, 0, -4, 17, 3, 8, 12, 10])  
keys_to_search = [8, 9, 21]  
  
for key in keys_to_search:  
    pos = binarySearch(list1, key)  
    if pos:  
        print(f"Number {key} is found at position {pos}")  
    else:
```

```
print(f"Number {key} is not found in the list")
```

Sample Output:

```
Number 8 is found at position 6  
Number 9 is not found in the list  
Number 21 is found at position 10
```

5. Following is a list of unsorted/unordered numbers:

[50, 31, 21, 28, 72, 41, 73, 93, 68, 43, 45, 78, 5, 17, 97, 71, 69, 61, 88, 75, 99, 44, 55, 9]

(a) Use linear search to determine the position of 1, 5, 55 and 99 in the list. Also note the number of key comparisons required to find each of these numbers in the list.

(b) Use a Python function to sort/arrange the list in ascending order.

(c) Again, use linear search to determine the position of 1, 5, 55 and 99 in the list and note the number of key comparisons required to find these numbers in the list.

(d) Use binary search to determine the position of 1, 5, 55 and 99 in the sorted list. Record the number of iterations required in each case.

(a) Unsorted List Linear Search

1 not found; key comparisons: 24

5 found at position 13; key comparisons: 13

55 found at position 23; key comparisons: 23

99 found at position 21; key comparisons: 21

(b) Sorting the list: `lst = [50, 31, 21, 28, 72, 41, 73, 93, 68, 43, 45, 78, 5, 17, 97, 71, 69, 61, 88, 75, 99, 44, 55, 9]`

`lst.sort()` # Now the list is sorted in ascending order.

(c) Linear Search in Sorted List

1 not found; key comparisons: 24

5 found at position 1; key comparisons: 1

55 found at position corresponding to sorted index (varies; check sorted list); key comparisons: count till index

99 found at the last or near end; key comparisons: up to its index

(d) Binary Search Iterations

1: Not found; about 4-5 iterations

5: Found in 2-4 iterations based on list

55, 99: Found in 3-5 iterations

6. Write a program that takes as input the following unsorted list of English words:

[Perfect, Stupendous, Wondrous, Gorgeous, Awesome, Mirthful, Fabulous, Splendid, Incredible, Outstanding, Propitious, Remarkable, Stellar, Unbelievable, Super, Amazing].

(a) Use linear search to find the position of Amazing, Perfect, Great and Wondrous in the list. Also note the number of key comparisons required to find these words in the list.

(b) Use a Python function to sort the list.

(c) Again, use linear search to determine the position of Amazing, Perfect, Great and Wondrous in the list and note the number of key comparisons required to find these words in the list.

(d) Use binary search to determine the position of Amazing, Perfect, Great and Wondrous in the sorted list. Record the number of iterations required in each case.

(a) Linear Search in Unsorted List

"Amazing": position 16, comparisons: 16

"Perfect": position 1, comparisons: 1

"Great": not found, comparisons: 16

"Wondrous": position 3, comparisons: 3

(b) Sorting the list:
`words = ["Perfect", "Stupendous",
"Wondrous", "Gorgeous", "Awesome", "Mirthful",
"Fabulous", "Splendid", "Incredible", "Outstanding",`

```
"Propitious", "Remarkable", "Stellar", "Unbelievable",  
"Super", "Amazing"]  
words.sort()
```

(c) Linear Search in Sorted List

"Amazing": now position 1, comparisons: 1

"Perfect": new position (depends on sort), comparisons till index

"Great": not found; comparisons: 16

"Wondrous": new position (check sorted list); comparisons till index

(d) Binary Search Iterations

"Amazing": usually found in 1-2 iterations

"Perfect", "Wondrous": 2-4 iterations

"Great": not found; about 4 iterations

7. Estimate the number of key comparisons required in binary search and linear search if we need to find the details of a person in a sorted database having 2^{30} (1,073,741,824) records when details of the person being searched lies at the middle position in the database. What do you interpret from your findings?

Binary search: Only **1 comparison** (since the desired record is at the middle position).

Linear search: 536,870,912 comparisons (half the records, as it checks sequentially from start to middle).

Interpretation: Binary search is **vastly more efficient** than linear search for large sorted lists.

8. Use the hash function: $h(\text{element}) = \text{element} \% 11$ to store the collection of numbers: [44, 121, 55, 33, 110, 77, 22, 66] in a hash table. Display the hash table created. Search if the values 11, 44, 88 and 121 are present in the hash table, and display the search results.

Creating the hash table (size 11):

$44 \% 11 = 0 \rightarrow \text{index } 0: 44$

$121 \% 11 = 0 \rightarrow \text{collision at index } 0$

$55 \% 11 = 0 \rightarrow \text{collision at index } 0$

$33 \% 11 = 0 \rightarrow \text{collision at index } 0$

$110 \% 11 = 0 \rightarrow \text{collision at index } 0$

$77 \% 11 = 0 \rightarrow \text{collision at index } 0$

$22 \% 11 = 0 \rightarrow \text{collision at index } 0$

$66 \% 11 = 0 \rightarrow \text{collision at index } 0$

Hash table after insertions:

Index	Value
0	66
1	None
2	None

Index	Value
3	None
4	None
5	None
6	None
7	None
8	None
9	None
10	None

Search results:

11: $11 \% 11 = 0$, but `hashTable[0] = 66`, so **not found**

44: $44 \% 11 = 0$, `hashTable[0] = 66`, **not found**

88: $88 \% 11 = 0$, `hashTable[0] = 66`, **not found**

121: $121 \% 11 = 0$, $\text{hashTable}[0] = 66$, **not found**

9. Write a Python program by considering a mapping of list of countries and their capital cities such as:

CountryCapital={'India':'New Delhi','UK':'London','France':'Paris','Switzerland': 'Berne','Australia': 'Canberra'}

Let us presume that our hash function is the length of the Country Name. Take two lists of appropriate size: one for keys (Country) and one for values (Capital). To put an element in the hash table, compute its hash code by counting the number of characters in Country, then put the key and value in both the lists at the corresponding indices. For example, India has a hash code of 5. So, we store India at the 5th position (index 4) in the keys list, and New Delhi at the 5th position (index 4) in the values list and so on. So that we end up with:

hash index = length of key - 1	List of Keys	List of Values
0	None	None
1	UK	London

hash index = length of key - 1	List of Keys	List of Values
2	None	None
3	Cuba	Havana
4	India	New Delhi
5	France	Paris
6	None	None
7	None	None
8	Australia	Canberra
9	None	None
10	Switzerland	Berne

Search results:

India: Hash index = $5-1 = 4 \rightarrow$ Capital is **New Delhi**

France: Hash index = $6-1 = 5 \rightarrow$ Capital is **Paris**

USA: Hash index = $3-1 = 2 \rightarrow$ No entry; result is **None (not found)**